

ELEN90098 REINFORCEMENT LEARNING FOR ENGINEERING

WORKSHOP 3: DEEP Q LEARNING

SEMESTER 2, 2025

This workshop introduces you to the Deep Q Learning algorithm. You will investigate the characteristics of these algorithms and evaluate their performance.

Submission: Each group must upload 2 files:

- (1) A pdf (.pdf) file containing your answers to Task 1. Note that:
 - Only one member of the group needs to upload this file;
 - The naming convention of the file is `Workshop3-<LastName1>-<LastName2>.pdf` for a group whose members have last names, LastName1 and LastName2.
- (2) A zip (.zip) file compressing your Q Learning code, with the (TODO) blanks all filled. Note that:
 - Only one member of the group needs to upload this file;
 - The naming convention of the file is `Workshop3-Code-<LastName1>-<LastName2>.zip` for a group whose members have last names, LastName1 and LastName2.

Note: Task 1 needs to be answered in this week's submission. The Final Report should include answers for both Task 1 and Task 2.

1. INTRODUCTION

Deep Q-Learning (DQN) extends the classical Q-Learning algorithm by using a neural network instead of a table to store Q-values. In traditional Q-Learning, we build a table mapping every state-action pair to a value, but this quickly becomes impossible when the state space is very large or continuous. DQN solves this by training a neural network to approximate $Q(s, a)$, allowing the agent to learn directly from raw inputs like images or high-dimensional features. This breakthrough enabled reinforcement learning to succeed in environments such as Atari games, where classical methods cannot scale.

This assignment requires you to implement and evaluate Deep Q learning for playing Atari games. The Deep Q Learning algorithm was covered in the lecture, and you will be provided with starter code. This assignment will be faster to run on a GPU, though it is possible to complete on a CPU as well.

We will test the algorithms on the `CartPole` environment. Up next, Section 2 gives an overview of the environments and a review of Deep Q Learning. Section 3 is the tasks to be done this week.

2. REVIEW

2.1. Environment: Cart Pole. A pole is attached by an un-actuated joint to a cart, which moves along a frictionless track. The pendulum is placed upright on the cart, and the goal is to balance the pole by applying forces in the left and right directions on the cart. ([Gymnasium website](#))

Action Space The action is an `ndarray` with shape `(1,)` which can take values 0,1 indicating the direction of the fixed force the cart is pushed with.

- 0: Push the cart to the left
- 1: Push the cart to the right

Observation Space

The observation is an `ndarray` with shape `(4,)` with the values corresponding to the following positions and velocities:

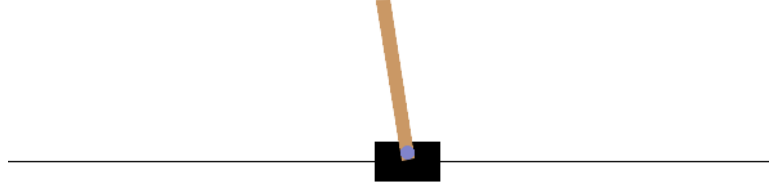


FIGURE 1. CartPole-v1

Idx	Observation	Min	Max
0	Cart Position	-4.8	4.8
1	Cart Velocity	$-\infty$	∞
2	Pole Angle	≈ -0.418 rad (-24°)	≈ 0.418 rad (24°)
3	Pole Angular Velocity	$-\infty$	∞

The cart's x-position (index 0) can take values between $(-4.8, 4.8)$, but the episode terminates if the cart leaves the $(-2.4, 2.4)$ range.

The pole angle can be observed between $(-0.418, 0.418)$ radians (or $\pm 24^\circ$), but the episode terminates if the pole angle is not in the range $(-0.2095, 0.2095)$ (or $\pm 12^\circ$).

Rewards

Since the goal is to keep the pole upright for as long as possible, by default, a reward of +1 is given for every step taken, including the termination step. The default reward threshold is 500 due to the time limit on the environment.

Termination

The episode ends if any one of the following occurs:

- Pole Angle is greater than $\pm 12^\circ$
- Cart Position is greater than ± 2.4 (center of the cart reaches the edge of the display)
- Episode length is greater than 500

2.2. Deep Q Learning. Deep Q-Learning (DQN) combines Q-Learning with deep neural networks to handle high-dimensional environments. Instead of using a Q-table, DQN approximates the action-value function $Q(s, a)$ with a parameterized neural network. Two key innovations stabilize learning:

- **Experience Replay:** Instead of learning from the most recent transition only, DQN stores past experiences in a **replay buffer** and samples **batches** randomly. This breaks correlations between consecutive samples and improves data efficiency.
- **Target Networks:** A separate copy of the Q-network, updated periodically, is used to compute target values. This reduces oscillations and stabilizes training.

These improvements allow DQN to scale reinforcement learning to challenging problems, bridging the gap between simple tabular algorithms and deep function approximators.

3. TASKS

File overview and Implementation. The starter code is provided with blanks marked as (TODO) for you to complete in order to run the algorithm. Move the code under the workshop folder `rl-workshop` that you have created.

The files are organized as follows:

- DQN/
 - `run_dqn.py`
 - `dqn/`
 - * `agent.py`
 - * `network.py`
 - `utils/`
 - * `log_utils.py`
 - * `logger.py`
 - * `util.py`

Your work is to complete the (TODO) blanks in `run_dqn.py` and `dqn/agent.py` to implement Deep Q Learning. Before running, navigate to the workshop folder:

```
cd path/to/rl-workshop
```

The starter code automatically logs results to TensorBoard. To visualize training curves, run:

```
tensorboard --logdir ./logs
```

Then open `localhost:6006` in your browser.

—

Task 1: CartPole (7 points). This task introduces you to the core components of DQN:

- Implementing the action selection strategy.
- Sampling batches from the replay buffer.
- Updating the Q-network using the sampled data.

Question 1 (2 points). What are the state space, action space, and reward structure of the `CartPole` environment? Are they finite or infinite? Discrete or continuous? Is the environment deterministic or stochastic?

Question 2 (3 points). Train a DQN agent on the `CartPole-v1` environment. The agent should be able to achieve the maximum reward of 500.

- Use TensorBoard to visualize training progress.
- Submit a plot showing evaluation returns (y -axis) versus environment steps (x -axis).
- Briefly comment on whether the learning curve indicates successful training.

Commands:

```
python DQN/run_dqn.py --env_name CartPole-v1 --log_video
```

—

Task 2. Deep Q Learning

Choose at least 2 of the 3 questions to answer

Question 1. Investigate how the learning rate α affects training outcomes. Train the agent with at least three different learning rates, e.g., 10^{-4} , 10^{-3} , 10^{-2} .

- 10^{-4}

```
python DQN/run_dqn.py --env_name CartPole-v1 --log_video --learning_rate
0.0001 --exp_name lr1e-4
```

- 10^{-3}

```
python DQN/run_dqn.py --env_name CartPole-v1 --log_video --learning_rate
0.001 --exp_name lr1e-3
```

- 10^{-2}

```
python DQN/run_dqn.py --env_name CartPole-v1 --log_video --learning_rate 0.01
--exp_name lr1e-2
```

Submit:

- A plot showing evaluation returns (y-axis) versus environment steps (x-axis) for each learning rate.

Discuss

- Which learning rate achieves faster convergence?
- Which one is more stable?
- Does too high a learning rate lead to divergence or unstable oscillations?
- Does too low a learning rate cause slow learning or poor final performance?

Question 2. Investigate how exploration schedules affect training outcomes. Here, exploration is handled by the ϵ -greedy policy, where ϵ decays over time. Compare 3 different schedules:

- $\epsilon = 0.1$ (constant ϵ)

```
python DQN/run_dqn.py --env_name CartPole-v1 --log_video --epsilon 0.1 --
epsilon_decay 1.0 --exp_name const_eps_0.1
```

- $\epsilon = 0.9^t$

```
python DQN/run_dqn.py --env_name CartPole-v1 --log_video --epsilon 1.0 --
epsilon_decay 0.9 --epsilon_min 0.02 --exp_name 0.9t
```

- $\epsilon = 0.995^t$

```
python DQN/run_dqn.py --env_name CartPole-v1 --log_video --epsilon 1.0 --
epsilon_decay 0.995 --epsilon_min 0.02 --exp_name 0.995t
```

Submit:

- A plot showing evaluation returns (y-axis) versus environment steps (x-axis) for all schedules.

Discuss

- How do different exploration schedules affect the agent's ability to explore and learn?
- Does keeping ϵ higher for longer lead to better performance, or does it slow down learning unnecessarily?
- What happens if ϵ is too small, or decays too quickly?

Question 3. Investigate how the replay buffer affects training outcomes. Train the agent with different values for memory size and batch size.

Compare at least 3 different values for memory size, e.g.,

- 10^5 steps

```
python DQN/run_dqn.py --env_name CartPole-v1 --log_video --memory_size 100000
--exp_name buffer1e5
```

- 10^4 steps

```
python DQN/run_dqn.py --env_name CartPole-v1 --log_video --memory_size 10000
--exp_name buffer1e4
```

- 10^3 steps

```
python DQN/run_dqn.py --env_name CartPole-v1 --log_video --memory_size 1000
--exp_name buffer1e3
```

Compare at least 2 different values for batch size, e.g.,

- 128 steps

```
python DQN/run_dqn.py --env_name CartPole-v1 --log_video --batch_size 128 --
exp_name batch128
```

- 32 steps

```
python DQN/run_dqn.py --env_name CartPole-v1 --log_video --batch_size 32 --
exp_name batch32
```

Submit:

- A plot showing evaluation returns (y-axis) versus environment steps (x-axis) for all cases.

Discuss

- How does memory size change the stability and smoothness of training curves?
- How does batch size affect convergence speed and final performance?
- In your experiments, which combination of memory size and batch size gave the best performance for CartPole?

—

Note: Task 1 must be answered this week. Task 2 must be answered in the final report. Answers to all tasks **must be included and discussed in your final report.**